

Linguagem e Técnicas de Programação

Professor Msc. Aparecido Vilela Junior
aparecido.vilela@unicesumar.edu.br

Dividir para Conquistar

Dividir um problema em subproblemas mais simples

Os passos para isso são:

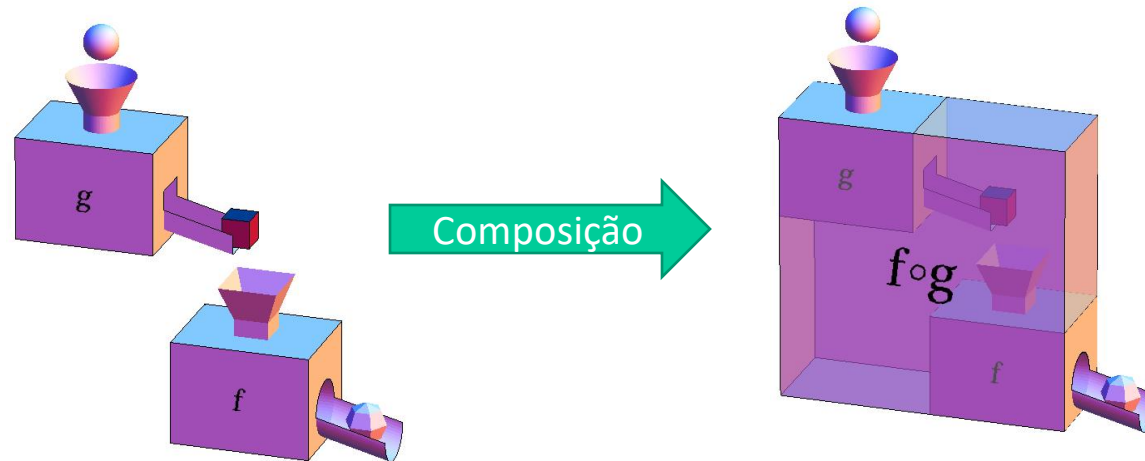
1. Divisão do problema em subproblemas;
2. Solução de cada um dos subproblemas;
3. Composição das soluções dos subproblemas para solucionar o problema original.

Chamado de programação modular

Programação Modular

Vantagens:

- Módulos podem ser escritos uma vez apenas e reutilizados sempre que necessário
- Módulos podem ser compostos para solucionar problemas cada vez complexos



Facilita a manutenção: um erro corrigido em um módulo reflete em todos os lugares onde esse módulo é utilizado;

Módulos em C - Funções

Função: conjunto de instruções para realizar uma ou mais tarefas que são agrupadas em uma mesma unidade e que pode ser referenciada

$$S = 1 + x - y + \frac{\sqrt{(x+y)^2}}{(x-y)^*2!} - \frac{\sqrt{(x+y)^3}}{(x-y)^*3!} + \dots$$

Funções em C

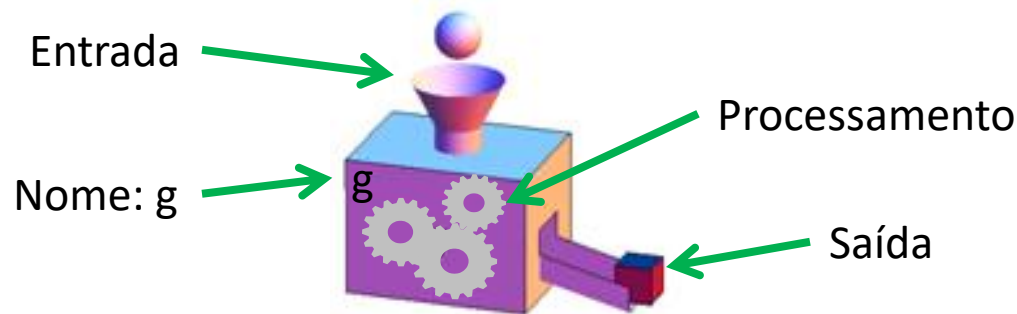
Para criação, é necessário informar:

Tipo das entradas (parâmetros): tipos de dados dos dados que são necessários para executar sua função (opcional);

Tipo da saída: tipo de dados do resultado do processamento (opcional);

Processamento: transforma as entradas na saída desejada;

Nome: um identificador (seguindo as regras para criação de identificadores para variáveis).



Exemplo da Sintaxe

Entradas e seus tipos:

1º parâmetro: float x

2º parâmetro: float a

3º parâmetro: float b

4º parâmetro: float c

Tipo de dados da
saída (retorno): float

```
float segundoGrau(float x, float a, float b, float c) {  
    float y;  
  
    y = a*x*x + b*x + c; //y = ax^2 + bx + c  
  
    return y;  
}
```

Processamento
“corpo da função”

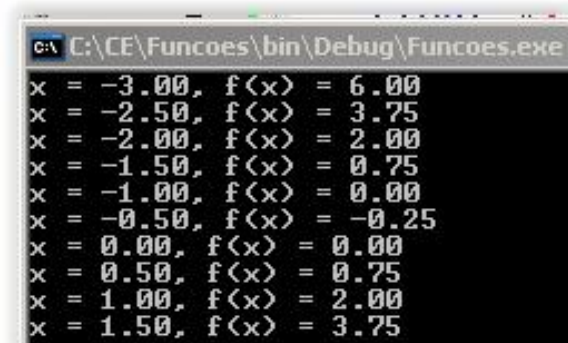
Nome da função: “segundoGrau”

Retornar para a saída o
resultado do processamento

Exemplo de Utilização

```
1  #include <stdio.h>
2
3  float segundoGrau(float x, float a, float b, float c) {
4      float y;
5
6      y = a*x*x + b*x + c; //y = ax^2 + bx + c
7
8      return y;
9  }
10
11  int main()
12  {
13      float x, y;
14
15      for (x=-3; x<2; x+=0.5) {
16          y = segundoGrau(x, 1, 1, 0); //y = x^2 + x
17
18          printf("x = %.2f, f(x) = %.2f\n", x, y);
19      }
20      return 0;
21  }
```

Nota: argumento é o nome dado aos valores passados para os parâmetros de uma função.



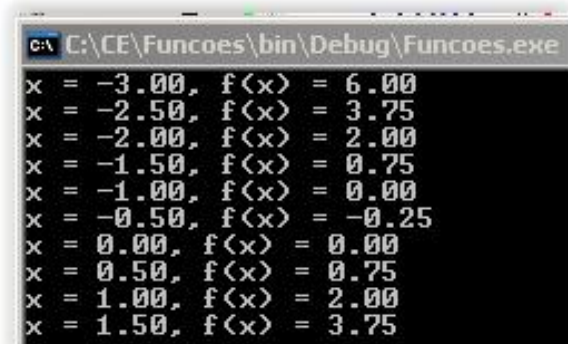
x	f(x)
-3.00	6.00
-2.50	3.75
-2.00	2.00
-1.50	0.75
-1.00	0.00
-0.50	-0.25
0.00	0.00
0.50	0.75
1.00	2.00
1.50	3.75

Exemplo de Utilização

O programa anterior equivale a

```
1  #include <stdio.h>
2
3  int main() ←
4  {
5      float x, y;
6
7      for (x=-3; x<2; x+=0.5) {
8          y = 1*x*x + 1*x + 0; //y = x^2 + x
9
10         printf("x = %.2f, f(x) = %.2f\n", x, y);
11     }
12     return 0;
13 }
14
15
16
```

Note que “main” é também uma função. Todo programa em C é uma função que deve retornar um código inteiro. Valor zero para este código indica que o programa terminou sem erros, qualquer outro valor indica um código de erro com significado definido pelo programador.



x	f(x)
-3.00	6.00
-2.50	3.75
-2.00	2.00
-1.50	0.75
-1.00	0.00
-0.50	-0.25
0.00	0.00
0.50	0.75
1.00	2.00
1.50	3.75

O comando return

Funções que retornam valores devem utilizar o comando **return**:

```
int segundos(int hora, int min) {  
    return 60 *(min + hora*60);  
}  
  
double porcentagem(double val, double tx) {  
    double valor = val*tx/100;  
    return valor;  
}
```

Obs.: O comando **return** pode aparecer em qualquer ponto do corpo da função, e uma vez atingido, a execução da função é terminada:

```
double porcentagem(double val, double tx) {  
    double valor = val*tx/100;  
    return valor;  
    printf("O valor foi calculado\n"); //<- Nunca será executado  
}
```

O comando return

Utilização:

return expressão;

Para executar este comando o programa:

Avalia expressão, obtendo um valor. Ex.: **return** (a*x*x+b*x+c);

Uma função que não tem valor para retornar: **void**

Uso do **return** é opcional:

```
void imprimeMenu() {  
    printf("1: Dilma\n");  
    printf("2: Aecio\n");  
    printf("3: Branco\n");  
    printf("4: Invalido\n");  
}
```

```
void imprimeMenu() {  
    printf("1: Dilma\n");  
    printf("2: Aecio\n");  
    printf("3: Branco\n");  
    printf("4: Invalido\n");  
    return;  
}
```

Variações

Algumas funções não precisam receber parâmetros.

Neste caso, a lista de parâmetros fica vazia, mas os parênteses ainda são obrigatórios:

```
void imprimeMenu() {  
    printf("1: Dilma\n");  
    printf("2: Aecio\n");  
    printf("3: Branco\n");  
    printf("4: Invalido\n");  
}
```

Chamada ou Invocação de Funções

Um programa em C, sempre inicia na função principal: `main()`;

Apenas declarar uma função não fará com que ela seja executada

Para que seja executada é necessário que ela seja **chamada** (invocada) -> fornecidos valores para os parâmetros

Quando chamada, o fluxo de controle do programa é **desviado** para a função e o código que está nela é executado;

Quando a função termina de ser executada, o fluxo de controle do programa **retorna** para a instrução logo após a chamada da função;

O valor de retorno da função pode ser capturado e armazenado em uma variável utilizando o comando de atribuição `'='`.

Voltado ao exemplo:

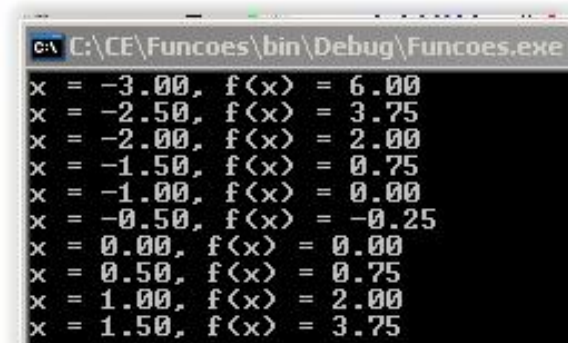
```
1  #include <stdio.h>
2
3  float segundoGrau(float x, float a, float b, float c) {
4      float y;
5
6      y = a*x*x + b*x + c; //y = ax^2 + bx + c
7
8      return y;
9  }
10
11 int main()
12 {
13     float x, y;
14
15     for (x=-3; x<2; x+=0.5) {
16         y = segundoGrau(x, 1, 1, 0); //y = x^2 + x
17         printf("x = %.2f, f(x) = %.2f\n", x, y);
18     }
19     return 0;
20 }
21
```

Declaração da função
"segundoGrau"

Chamada da função

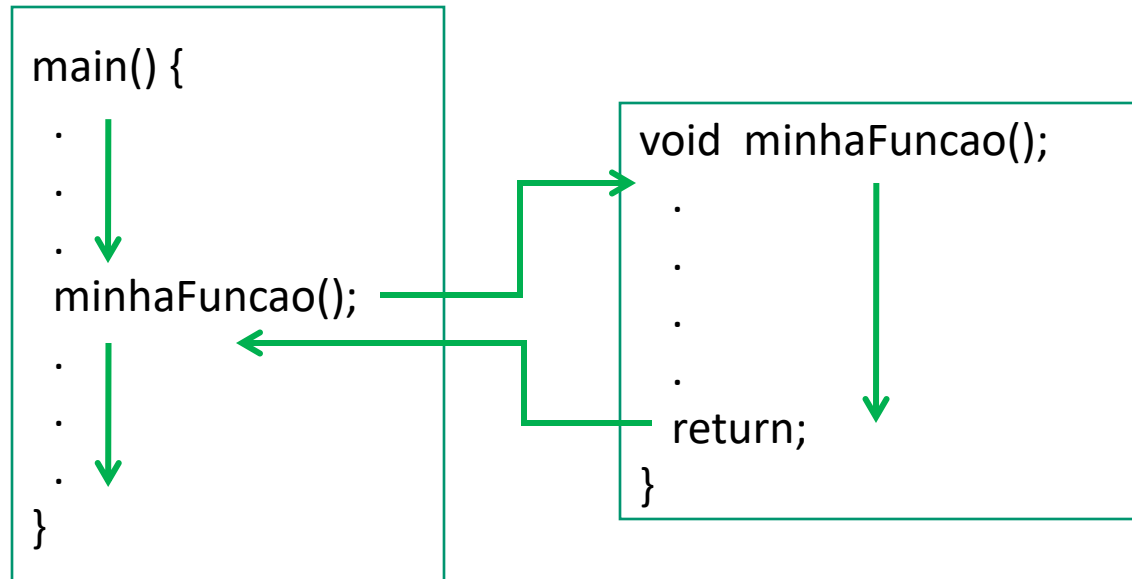
Passagem de valores para os parâmetros

Captura do valor retornado e
armazenamento da na variável y.

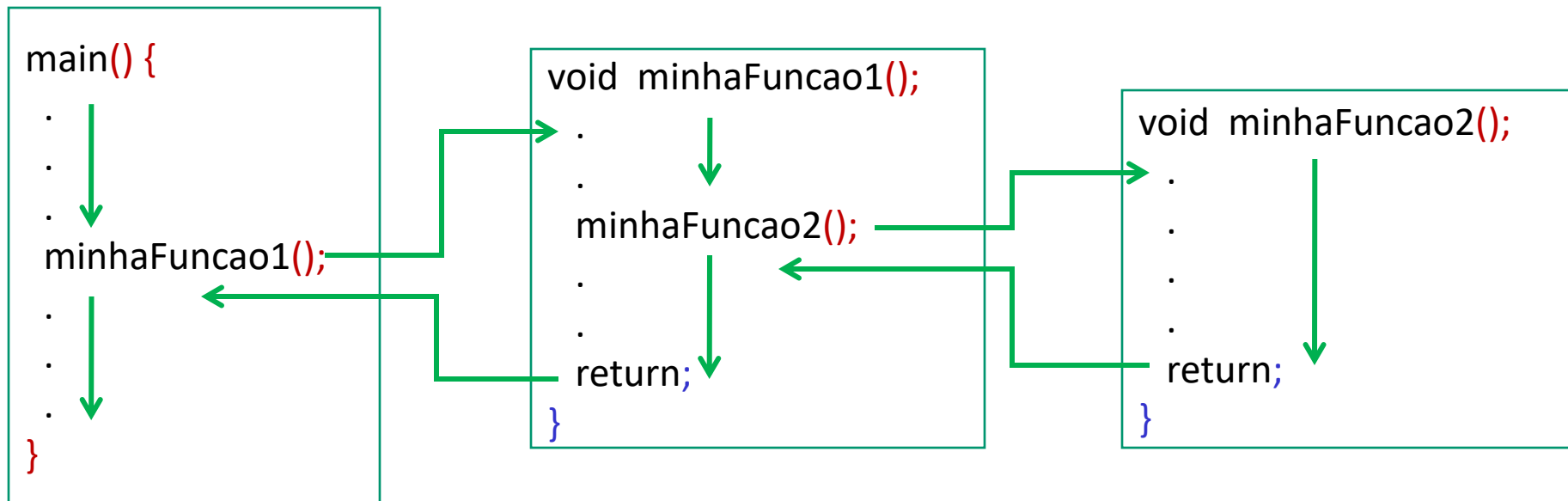
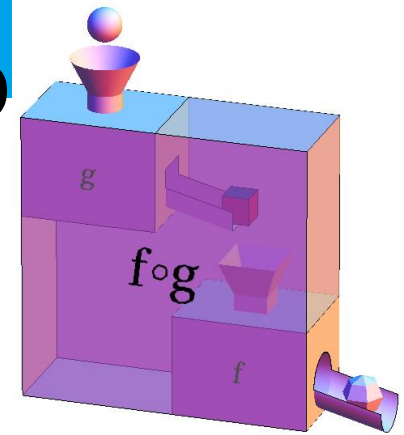


x	f(x)
-3.00	6.00
-2.50	3.75
-2.00	2.00
-1.50	0.75
-1.00	0.00
-0.50	-0.25
0.00	0.00
0.50	0.75
1.00	2.00
1.50	3.75

Desvio da Execução



Desvio da Execução



Escopo das Variáveis

Define a área do programa onde esta variável pode ser referenciada

Variáveis globais: declaradas fora das funções (inclusive fora da função **main**)

Podem ser referenciadas por todas as funções do programa abaixo do ponto onde foram declaradas

Variáveis locais: declaradas dentro de uma função (inclusive dentro da função **main**)

Só podem ser referenciadas dentro desta função

Variáveis Globais

Podem ser usadas em qualquer parte do código;

Existem durante todo o ciclo de vida do programa (ocupando memória);

Se não forem explicitamente inicializadas, são inicializadas para zero pelo compilador.

Normalmente declaradas no início do programa ou em arquivos do tipo header (*.h)

Declaradas uma única vez

Deve-se evitar o uso abusivo delas, pois:

Pode penalizar o consumo de memória;

Pode dificultar a legibilidade e manutenção do código (se pode ser acessada e alterada em qualquer lugar como encontrar onde está o erro?).

Variáveis Globais

```
1  #include <stdio.h>
2
3  int i; //Variável global
4
5  void incrementa() {
6      i++;
7  }
8
9  int main()
10 {
11     i = 0;
12     incrementa();
13     printf("Valor de i: %d", i);
14     return 0;
15 }
```

Variável global: declarada fora de qualquer função

Acessível em qualquer ponto do código após sua declaração

Variáveis Locais

Declaradas dentro de uma função

Só existem durante a execução da função -> só ocupam a memória durante a execução da função

Não são inicializadas automaticamente

São visíveis apenas dentro da função onde foram declaradas

Outras funções não podem referenciá-las

Parâmetros de funções podem ser vistos como variáveis locais

Variáveis Locais

```
1  #include <stdio.h>
2
3  void incrementa() {
4      int i = 0;
5      i++;
6  }
7
8  int main()
9  {
10     i = 0;
11     incrementa();
12     printf("Valor de i: %d", i);
13     return 0;
14 }
15
```

Variável local: declarada dentro de uma função

Não é acessível fora da função onde foi declarada.

Error: 'i' undeclared!

Parâmetros e Argumentos

Os parâmetros são nomes que aparecem na declaração de uma função:

```
void imprimir(int valor)
```

Os argumentos são expressões que aparecem na expressão de invocação da função:

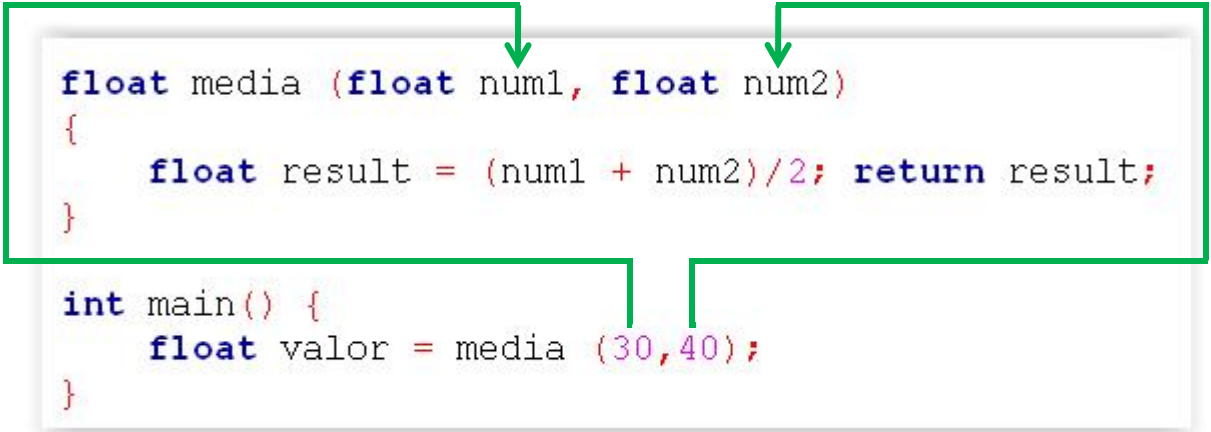
```
imprimir(10);
```

```
imprimir(8+2);
```

```
Imprimir(2*5);
```

Parâmetros e Argumentos

Quando uma função é chamada, os argumentos da chamada são copiados para os parâmetros (formais) presentes na assinatura da função:



```
float media (float num1, float num2)
{
    float result = (num1 + num2)/2; return result;
}

int main() {
    float valor = media (30,40);
}
```

The diagram illustrates the flow of data from the `main` function to the `media` function. A green box encloses the `media` function definition. Two green arrows point from the arguments `30` and `40` in the `media` call within `main` to the parameters `num1` and `num2` in the `media` function signature. Another green arrow points from the `media` function call back to the `main` function, indicating the return value.

Parâmetros são como variáveis locais da função (não é necessário declarar novamente)

Não se deve declarar variáveis locais com o mesmo nome de parâmetros

Escopo das Variáveis

Variáveis em escopos diferentes podem ter o mesmo nome, porém, referenciam endereços de memória diferentes!

```
1  #include <stdio.h>
2
3  void incrementa() {
4      int i = 0;
5      i++;
6  }
7
8  int main()
9  {
10     int i = 10;
11     incrementa();
12     printf("Valor de i: %d", i);
13     return 0;
14 }
15
```

Mesmo nome, porém
são variáveis distintas

Qual valor será impresso?

Escopo das Variáveis

Uma variável de escopo local, com o mesmo nome de uma variável com escopo global oculta (sobrepõe) a de escopo global.

```
1  #include <stdio.h>
2
3  int i = 0;
4
5  void incrementa() {
6      i++;
7      printf("Valor de i em incrementa: %d", i);
8  }
9
10 int main()
11 {
12     int i = 10;
13     incrementa();
14     printf("Valor de i na
15     return 0;
16 }
```

Referência a
variável global

A variável de
escopo local na
main, sobrepõe a
de escopo global

Referência a
variável local

Quais valores serão impressos?

Ordem da Definição de Funções

Onde uma função deve ser definida?

Antes da **main**; ou

Depois da **main**, desde que sua assinatura seja declarada antes da **main**.

A assinatura de uma função deve indicar:

seu nome;

Os tipos das entradas;

O tipo da saída.

Ex.: Função “segundos”:

Transforma horas e minutos
em segundos.

```
int segundos(int hora, int min) {  
    return 60 * (min + hora*60);  
}
```

Assinatura da função “segundos”:

O nome dos parâmetros é opcional:

```
int segundos(int, int);
```

```
int segundos(int hora, int min)
```

Ordem da Definição de Funções

Onde uma função deve ser definida?

Antes da **main**:

```
int segundos(int hora, int min) {  
    return 60 *(min + hora*60);  
}  
  
int main() {  
    int minutos, hora, seg ;  
    printf("Digite a hora:minutos\n");  
    scanf ("%d:%d",&hora,&minutos) ;  
    seg = segundos(hora,minutos);  
    printf("\n%d:%d tem %d segundos.",hora,minutos,seg);  
    return 0 ;  
}
```

Ordem da Definição de Funções

Onde uma função deve ser definida?

Depois da **main** com declaração prévia da assinatura:

```
int segundos(int, int); ←  
  
int main() {  
    int minutos, hora, seg ;  
    printf("Digite a hora:minutos\n");  
    scanf ("%d:%d",&hora,&minutos) ;  
    seg = segundos(hora,minutos); ←  
    printf("\n%d:%d tem %d segundos.",hora,minutos,seg);  
    return 0 ;  
}  
  
int segundos(int hora, int min) {  
    return 60 *(min + hora*60);  
} ←
```

A regra básica é que o compilador precisa encontrar a definição de uma função ou sua assinatura antes de encontrar sua chamada

Atividade 1

Atividade vista em aulas anteriores:

1. Escreva um algoritmo que lê 50 números inteiros e em seguida mostra a soma de todos os ímpares lidos.
2. Altere o algoritmo anterior para que ele considere apenas a soma dos ímpares que estejam entre 100 e 200.
3. Construa um algoritmo que leia um conjunto de 20 números inteiros e mostre qual foi o maior e o menor valor fornecido.
4. Altere o programa anterior para que ele não permita a entrada de valores negativos.

Atividade 1

Faça um programa que leia dois valores inteiros x e y entre 0 e 1000. Encontre o maior entre eles e imprima:

O percentual do menor em relação ao maior

O modulo da diferença entre o maior e o menor

Altere o programa anterior para que utilize três funções:

- a) **scanIntIntervalo**: Função para ler inteiros do teclado garantido que eles estejam dentro de um intervalo pré-determinado;
- b) **percentual**: Função para calcular o percentual: $100 * \text{valor} / \text{total}$
- c) **absdif**: Função que retorna o valor absoluto da diferença entre dois números reais.

Atividade 1

Um centro materno-infantil deseja criar um programa para recomendar aos médicos sobre o tipo de parto a ser adotado. O mecanismo de recomendação utiliza o peso do feto e quantidade de semanas de gestação para sugerir o tipo de parto mais indicado. Desenvolva um programa na linguagem C, o qual deverá:

Ler o peso do feto em gramas e a quantidade de semanas da gestação. Caso o peso do feto seja inferior que 100 gramas ou a quantidade de semanas menor que 28, o programa deverá exibir a mensagem "Parto não deverá ser realizado, reavaliar clinicamente" e encerrar a execução.

Caso contrário, o programa deverá calcular a quantidade de meses (considerar 4 semanas para cada mês) do feto e exibir uma das recomendações abaixo:

Peso superior a 2.500 gramas e com mais de 7 meses: "Parto normal";

Peso superior a 2.500 gramas e abaixo ou com 7 meses: "Parto Cesariana";

Entre 2.000 gramas e 1.500 gramas e acima de 9 meses: "Parto normal";

Qualquer outra combinação, "Parto Cesariana".

Atividade 2

Um número perfeito é um número inteiro para o qual a soma de todos os seus divisores positivos próprios (excluindo ele mesmo) é igual ao próprio número. Por exemplo, o número 6 é um número perfeito, pois: $6 = 1 + 2 + 3$. O próximo número perfeito é o 28, pois: $28 = 1 + 2 + 4 + 7 + 14$.

A matemática ainda não sabe se a quantidade de números perfeitos pares é ou não finita. Não se sabe também se existem números perfeitos ímpares. Escreva um programa em C que realize as seguintes operações:

- a) Leia um número inteiro e verifique se ele é par, caso seja ímpar obrigue o usuário a digitar outro número até que um número par seja digitado;
- b) Verifique se o número digitado é perfeito e imprima uma mensagem tela indicando se o número digitado é perfeito ou não.

Atividade 3 - Fatorial

Na matemática, o fatorial de um número natural n , representado por $n!$, é o produto de todos os inteiros positivos menores ou iguais a n .

Construa uma função que receba como parâmetro n e retorne o fatorial de n :

```
int fat(int n) //Recebe  $n$  como parâmetro e retorna  $n!$ 
```

Obs.: Utilize laço e variáveis locais!

Atividade 4 - Fibonacci

Na matemática, a sequência de Fibonacci, é uma sequência de números inteiros, começando normalmente por 0 e 1, na qual, cada termo subsequente (numero de Fibonacci) corresponde a soma dos dois anteriores.

A sequência recebeu o nome do matemático italiano Leonardo de Pisa, mais conhecido por Fibonacci, que descreveu, no ano de 1202, o crescimento de uma população de coelhos, a partir desta.

Tal sequência já era no entanto, conhecida na antiguidade.

Os números de Fibonacci são, portanto, os números que compõem a seguinte sequência:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377...

Atividade 4 - Fibonacci

Assim, o número de Fibonacci F_n para $n > 0$ é definido da seguinte maneira:

$$F_1 = 1$$

$$F_2 = 1$$

$$F_n = F_{n-1} + F_{n-2} \text{ para } n > 2.$$

Escreva uma função que retorne o número relativo a ao valor na sequência de Fibonacci na posição n :

```
int fib(int n) //Recebe  $n$  como parâmetro e retorna  $F_n$ 
```

Obs.: Utilize laço e variáveis locais!